MARS-OPS — Team Playbook

Programming II · Lab 5 · Autonomous Mars Colony Management & Emergency Simulator Team of 3 (Members A, B, C) · Java 17 · Version 1.1

AI assistant reading this file? Go straight to Appendix A before doing anything.

---

1. Start here (5 minutes)

▶ YOUR FIRST 5 MINUTES

1. **Find your letter** (A, B or C) in the table in Section 2. That is your part — every file you own is listed there.
2. **Do Day 0 together** (Section 4). About 45 minutes, once.
3. **Open your section** — A → Section 8, B → Section 9, C → Section 10 — and follow its **"Do this first"** list.
4. **Work alone, in parallel.** Every signature you need from the other two is already written in this document. Nobody waits for anybody.
5. **Test as you go** with your own test file. **Push small and often.** Tick your **"Done when"** list.
6. **Last stretch:** UML (Section 12) → README (Section 13) → Submit (Section 15).

★ THE 4 GOLDEN RULES

1. **Edit only your own files.** (Then Git never has conflicts.)
2. **Signatures are frozen.** Copy class, field and method names exactly as written here.
3. **`git pull` before you start, `git pull` + `git push` when you finish.**
4. **Everybody must be able to explain everything** — the discussion grades every member on the whole system.

Where to find things

I want to…	Go to
Know which files are mine	Section 2
Set up GitHub and compile	Section 4
See the rules we all agreed on (resources, missions, emergencies)	Section 6
Copy the shared code everyone pastes on Day 0	Section 7
Build my part	Section 8 (A) · 9 (B) · 10 (C)
Get the numbers for tests and demos	Section 11
Draw the UML diagrams	Section 12
Write my README part	Section 13
Write Javadoc and clean code	Section 14
Submit and prepare for the discussion	Section 15

---

2. Team and ownership

Who is who (fill in)

Member	Name	Part
A	(write name)	**Things** — agents, facilities, environment, report
B	(write name)	**Brain** — missions, resources, colony rules
C	(write name)	**Crisis & Control** — emergencies, simulation, Main

Who owns which files (you edit ONLY your files)

All packages live under `src/marsops/`.

Package	Files	Owner
`common`	Capability, Skill, ResourceType, MissionType, MissionState, EnvironmentCondition, EmergencyType, EmergencyState, FacilityStatus, RobotCondition, Location, MartianSurface, Equipment, ResourceRequirement	Shared code — pasted once on Day 0 (Section 7), then **frozen**
`tests`	Check	Shared code (Section 7)
`agents`	Agent, CrewMember, Scientist, Engineer, MedicalOfficer, Robot, ExplorationRover, RepairRobot, Drone	**A**
`facilities`	Facility, ResourceInventory, Habitat, LifeSupportSystem, Laboratory, StorageFacility, PowerStation, CommunicationStation	**A**
`environment`	Environment	**A**
`report`	ColonyReport	**A**
`tests`	AgentsFacilitiesTests	**A**
`missions`	Mission, AbstractMission, ExplorationMission, RepairMission, ResearchMission, RescueMission	**B**
`colony`	MarsColony, SubmissionResult, ResourceManager, AllocationPolicy, StorageFirstPolicy	**B**
`tests`	ColonyMissionsTests	**B**
`emergencies`	Emergency, OxygenLeak, Fire, RoverBreakdown, MedicalEmergency, EmergencyManager	**C**
`simulation`	Simulation	**C**
`marsops` (root)	Main	**C**
`tests`	EmergencySimulationTests	**C**

Folder structure

```
StudentIDs-Prog-II-Lab5/
├── README.md                          (assembled from Section 13)
├── MARS-OPS-MASTER.pdf                (this playbook)
├── uml/                               (diagram images/PDF + editable .drawio sources)
├── .gitignore
└── src/marsops/
    ├── Main.java                                  (C)
    ├── common/…                                   (shared code)
    ├── agents/…        facilities/…               (A)
```

```
    ├── environment/…   report/…              (A)
    ├── missions/…      colony/…              (B)
    ├── emergencies/…   simulation/…          (C)
    └── tests/Check.java, *Tests.java
```

---

3. What we are aiming for

Part	Marks	What we must deliver
Lab implementation	4	Working Java program · all 5 demo scenarios pass · README complete · clean ZIP
UML diagrams	2 (bonus)	5 diagram types: Class, Object, State Machine, Sequence (3 of them), Use Case
Emergency handling	1 (bonus)	All 4 emergencies work · oxygen leak follows the exact rules (double loss if unresolved)
Total	**7**	4 regular + 3 bonus

UML has not been taught yet — **no problem**. Section 12 teaches it from zero and already decides what goes into every diagram.

---

4. Day 0 — set up together (about 45 minutes)

4.1 GitHub, the simplest possible way

Everybody works directly on `main`. It is safe because **every file has exactly one owner**, so Git never sees two people changing the same file.

One person (the "repo owner") does this once 1. On github.com: **New repository** → name `mars-ops` → **Private** → *Create*. 2. *Settings* → *Collaborators* → *Add people* → add the other two. 3. On your computer: `git clone https://github.com/<owner>/mars-ops.git cd mars-ops` 4. Create the folders from Section 2, paste the shared code (Section 7) and the `.gitignore` below, then: `git add . git commit -m "Day 0: shared code and playbook" git push`

The other two do this once

```
git clone https://github.com/<owner>/mars-ops.git
cd mars-ops
git config user.name "Your Name"
git config user.email "you@example.com"
```

Your daily loop — memorise these 4 steps

```
git pull                      # 1. get the others' work FIRST
# ...write your code...
git add .                     # 2. stage your files
git commit -m "A: add Facility and ResourceInventory"   # 3. save (who + what)
git pull                      # 4a. get anything new again
git push                      # 4b. upload
```

> **If `git push` says "rejected":** someone pushed before you. Run `git pull`, then `git push` again.
> **If Git says `CONFLICT`** (should not happen if you only edit your own files): 1. Open the file named in the message and search for `<<<<<<<`. 2. Keep the lines you want. Delete the lines `<<<<<<<`, `=======`, `>>>>>>>`. 3. Run `git add .` → `git commit -m "Fix merge conflict"` → `git push`. 4. Tell the group chat which file it was.

Never: `git push --force` · edit another member's file · commit the `out/` folder. **Prefer buttons?** GitHub Desktop or the VS Code *Source Control* tab do the same 4 steps (Pull → Commit → Pull → Push).

`.gitignore`:

```
out/
sources.txt
*.class
.idea/
.vscode/
```

4.2 Compile and run

Linux / macOS:

```
find src -name "*.java" > sources.txt
javac -d out @sources.txt
java -cp out marsops.Main
java -cp out marsops.tests.AgentsFacilitiesTests
```

Windows (cmd):

```
dir /s /b src\*.java > sources.txt
javac -d out @sources.txt
java -cp out marsops.Main
```

Test classes: `marsops.tests.AgentsFacilitiesTests`, `marsops.tests.ColonyMissionsTests`, `marsops.tests.EmergencySimulationTests`.

4.3 Stub phase (each member alone, 30–45 minutes)

For **every file you own**: create it and copy the fields, constructors and method signatures from your section. Give each body `throw new UnsupportedOperationException("TODO");` (or `return null;` / `return false;` where a throw would stop it compiling). Compile, `git pull`, `git push`.

> ✔ **CHECKPOINT:** when all three have pushed, `javac` must succeed for everybody. From now on everyone fills in their own bodies — nobody is blocked.

---

5. Team rules

1. **Change a signature?** Tell the group chat → agree → the human who owns this playbook updates it → everybody pulls the new version. Code and playbook must always match.
2. **Never push code that does not compile.**
3. **Model classes never print.** Only `Main`, `ColonyReport` and tests print. Model classes return Strings or call `colony.log(...)`.
4. **No randomness.** Emergencies and environment changes are scheduled or triggered by hand, so demos and tests are repeatable.

5. **Waiting for a teammate's code?** Write a tiny local *fake* inside your own test file (never commit a fake into someone else's package).
6. **Help each other by reviewing,** not by editing each other's files.

---

6. Rules we all follow (copy into the README as "Assumptions")

ID	Decision
D-01	**Resource allocation policy:** `StorageFirstPolicy` — take from `StorageFacility` objects first, then from other operational facilities in the order they were added to the colony.
D-02	Colony-wide total = sum of `getAvailable(type)` over facilities. Facilities that are not `OPERATIONAL` (DAMAGED, UNAVAILABLE, DESTROYED) contribute 0.
D-03	Consumption is **all-or-nothing**. Mission resources are consumed **when the mission is accepted** and are **not refunded** if it fails later. Rejected missions consume nothing.
D-04	**Time:** the first `tick` after acceptance moves the mission `ASSIGNED → IN_PROGRESS` and that same tick counts as unit 1. A 3-unit mission completes on the 3rd tick.
D-05	**Mission states:** `CREATED → ASSIGNED → IN_PROGRESS → COMPLETED`; `CREATED/ASSIGNED/IN_PROGRESS → FAILED`. `COMPLETED` and `FAILED` are final; failed missions are never retried (create a new `Mission`).
D-06	**Environment vs. missions already running:** `DUST_STORM` fails active surface (rover) missions immediately; `COMMUNICATION_DISRUPTION` fails active missions that need communication (Rescue); other missions continue. Applied the moment `changeEnvironment(...)` is called.
D-07	A mission is accepted only if every agent can perform it, every agent is available, and together they cover all required capabilities.
D-08	An agent is **usable** unless: human health ≤ 20 or fatigue = 100; robot DAMAGED or battery = 0. **Available** = usable AND not assigned.
D-09	Battery drain per tick while assigned: Rover 5, Drone 8, RepairRobot 3. Idle robots recharge +1 (max 100). Human fatigue: +10 per tick while assigned (max 100), −5 per tick when idle (min 0).
D-10	**Responders needed** (one distinct available agent per capability): OXYGEN_LEAK = ENGINEERING + ROBOTIC_REPAIR · FIRE = ENGINEERING + MEDICAL · ROVER_BREAKDOWN = ENGINEERING + ROBOTIC_REPAIR · MEDICAL_EMERGENCY = MEDICAL.
D-11	**Response resources:** OXYGEN_LEAK: 2 SPARE_PARTS + 5 BATTERY_POWER · FIRE: 10 WATER + 1 MEDICAL_SUPPLIES + 3 BATTERY_POWER · ROVER_BREAKDOWN: 2 SPARE_PARTS + 5 BATTERY_POWER · MEDICAL_EMERGENCY: 2 MEDICAL_SUPPLIES + 5 OXYGEN.
D-12	Responders must be available (usable and not in the active mission). Dispatch moves them to the emergency location; they **stay there** afterwards. Unresolved emergencies are never retried automatically.
D-13	**Oxygen leak:** initial deduction on detection; if repaired this is the total loss; if UNRESOLVED the same amount is deducted again (never below 0). One-time deduction, no continuous drain.
D-14	**Fire:** lab becomes `UNAVAILABLE`, all its equipment is damaged, lab burns 5 oxygen from its own stock, every agent in the lab takes 30 damage. Resolved → lab `DAMAGED` (a Repair mission can restore it), fire out, people in the lab recover 20. Unresolved → lab `DESTROYED` (its stock no longer counts).
D-15	**Rover breakdown:** rover becomes DAMAGED and stays where it broke down; if it is part of the active mission, that mission fails. Resolved → rover repaired and sent home.
D-16	**Medical emergency:** patient takes `severity` damage on detection; resolved → patient recovers 40. Unresolved if no medic, no supplies, or the facility is not operational. The patient can never be their own responder.
D-17	Emergencies do not interrupt the active mission (except D-15 for the broken rover).
D-	Facility `capacity` = maximum total resource units the facility can store.

```
18
```

7. Shared code — paste once on Day 0, then FROZEN

The repo owner pastes these files on Day 0. Nobody edits them afterwards.

```
// ===== src/marsops/common/Capability.java =====
package marsops.common;
/** What an agent can DO. Missions and emergencies ask for capabilities, never for concrete classes. */
public enum Capability { RESEARCH, ENGINEERING, ROBOTIC_REPAIR, MEDICAL, SURFACE_TRAVEL, AERIAL_RECON,
VEHICLE }

// ===== src/marsops/common/Skill.java =====
package marsops.common;
/** Skills of human crew members. */
public enum Skill { GEOLOGY, BIOLOGY, CHEMISTRY, ENGINEERING, MEDICINE }

// ===== src/marsops/common/ResourceType.java =====
package marsops.common;
public enum ResourceType { OXYGEN, WATER, FOOD, FUEL, BATTERY_POWER, MEDICAL_SUPPLIES, SPARE_PARTS }

// ===== src/marsops/common/MissionType.java =====
package marsops.common;
public enum MissionType { EXPLORATION, REPAIR, RESEARCH, RESCUE }

// ===== src/marsops/common/MissionState.java =====
package marsops.common;
public enum MissionState { CREATED, ASSIGNED, IN_PROGRESS, COMPLETED, FAILED }

// ===== src/marsops/common/EnvironmentCondition.java =====
package marsops.common;
public enum EnvironmentCondition { NORMAL, DUST_STORM, COMMUNICATION_DISRUPTION }

// ===== src/marsops/common/EmergencyType.java =====
package marsops.common;
public enum EmergencyType { OXYGEN_LEAK, FIRE, ROVER_BREAKDOWN, MEDICAL_EMERGENCY }

// ===== src/marsops/common/EmergencyState.java =====
package marsops.common;
public enum EmergencyState { DETECTED, RESOLVED, UNRESOLVED }

// ===== src/marsops/common/FacilityStatus.java =====
package marsops.common;
public enum FacilityStatus { OPERATIONAL, DAMAGED, UNAVAILABLE, DESTROYED }

// ===== src/marsops/common/RobotCondition.java =====
package marsops.common;
public enum RobotCondition { OPERATIONAL, DAMAGED }

// ===== src/marsops/common/Location.java =====
package marsops.common;
/** A place an agent can be: a Facility or the Martian surface. */
public interface Location { String getName(); }

// ===== src/marsops/common/MartianSurface.java =====
package marsops.common;
/** The open Martian surface (single shared instance). */
public final class MartianSurface implements Location {
    public static final MartianSurface INSTANCE = new MartianSurface();
    private MartianSurface() { }
    @Override public String getName() { return "Martian Surface"; }
    @Override public String toString() { return getName(); }
}

// ===== src/marsops/common/Equipment.java =====
package marsops.common;
/** A piece of equipment (e.g. "Spectrometer"). Can be damaged and repaired. */
public class Equipment {
    private final String name;
    private boolean functional = true;
    public Equipment(String name) {
```

```
        if (name == null || name.isBlank()) throw new IllegalArgumentException("name required");
        this.name = name;
    }
    public String getName() { return name; }
    public boolean isFunctional() { return functional; }
    public void damage() { functional = false; }
    public void repair() { functional = true; }
    @Override public String toString() { return name + (functional ? "" : " (damaged)"); }
}

// ===== src/marsops/common/ResourceRequirement.java =====
package marsops.common;
import java.util.Collections;
import java.util.EnumMap;
import java.util.Map;
/** "How much of each resource an operation needs", e.g. new ResourceRequirement().with(SPARE_PARTS,
2).with(BATTERY_POWER, 5). */
public class ResourceRequirement {
    private final Map<ResourceType, Integer> amounts = new EnumMap<>(ResourceType.class);
    public ResourceRequirement with(ResourceType type, int amount) {
        if (type == null || amount <= 0) throw new IllegalArgumentException("type required and amount must
be > 0");
        amounts.put(type, amount);
        return this;
    }
    public int get(ResourceType type) { return amounts.getOrDefault(type, 0); }
    public Map<ResourceType, Integer> asMap() { return Collections.unmodifiableMap(amounts); }
    public boolean isEmpty() { return amounts.isEmpty(); }
    @Override public String toString() { return amounts.toString(); }
}

// ===== src/marsops/tests/Check.java =====
package marsops.tests;
/** Tiny test helper (no JUnit needed). Every test file calls Check.* and ends with Check.summary(). */
public final class Check {
    private static int passed = 0;
    private static int failed = 0;
    private Check() { }
    public static void section(String title) { System.out.println("\n== " + title + " =="); }
    public static void isTrue(String name, boolean condition) {
        if (condition) { passed++; System.out.println("  PASS  " + name); }
        else { failed++; System.out.println("  FAIL  " + name); }
    }
    public static void equal(String name, Object expected, Object actual) {
        boolean ok = java.util.Objects.equals(expected, actual);
        isTrue(ok ? name : name + "  [expected=" + expected + ", actual=" + actual + "]", ok);
    }
    /** Passes if the text contains the part, ignoring upper/lower case. */
    public static void contains(String name, String text, String part) {
        boolean ok = text != null && text.toLowerCase().contains(part.toLowerCase());
        isTrue(ok ? name : name + "  [text=\"" + text + "\" should contain \"" + part + "\"]", ok);
    }
    public static void throwsException(String name, Class<? extends Throwable> type, Runnable action) {
        try { action.run(); isTrue(name + "  [no exception thrown]", false); }
        catch (Throwable t) { isTrue(name, type.isInstance(t)); }
    }
    public static void summary() {
        System.out.println("\nRESULT: " + passed + " passed, " + failed + " failed");
        if (failed > 0) System.exit(1);
    }
}
```

8. MEMBER A — "Things"

> ▶ **YOUR MISSION (A)** Build the *things* of the colony: the **people and robots** (agents), the **buildings** (facilities), the **weather** (environment) and the **status report**. **Others need your facilities first — push them early. You also draw:** Class diagram + Use Case diagram · **You also write:** README sections R1, R2, R3. **Folders:** `agents`, `facilities`, `environment`, `report`, and `tests/AgentsFacilitiesTests.java`.

8.1 Do this first

- ☐ **1. Early push:** `ResourceInventory`, `Facility`, `StorageFacility`, `Habitat`, `LifeSupportSystem`, `Laboratory`, `PowerStation`, `CommunicationStation`, `Environment`.
- ☐ **2.** `Agent`, `CrewMember`, `Scientist`, `Engineer`, `MedicalOfficer`.
- ☐ **3.** `Robot`, `ExplorationRover`, `RepairRobot`, `Drone`.
- ☐ **4.** `ColonyReport`.
- ☐ **5.** `AgentsFacilitiesTests`, then the two UML diagrams.

8.2 `marsops.agents`

```
package marsops.agents;

/** Anything that can be assigned to missions and emergencies (humans and robots). */
public interface Agent {
    String getId();
    String getName();
    Location getLocation();
    void setLocation(Location location);
    Location getHomeLocation();                         // where it starts and returns to
    Set<Capability> getCapabilities();                  // unmodifiable
    default boolean hasCapability(Capability c) { return getCapabilities().contains(c); }

    String getUnusableReason();                         // null when usable, otherwise a clear sentence
    default boolean isUsable() { return getUnusableReason() == null; }
    boolean isAssigned();                               // true while part of a mission / emergency response
    void assignToMission();                             // throws IllegalStateException if already assigned
    void release();                                     // sets assigned = false
    default boolean isAvailable() { return isUsable() && !isAssigned(); }

    /** null if the agent CAN perform the mission; otherwise a sentence, e.g. "Dr. Salma does not possess
the required capability". */
    String getCannotPerformReason(Mission mission);
    default boolean canPerform(Mission mission) { return getCannotPerformReason(mission) == null; }

    boolean canHandle(EmergencyType type);              // can this agent respond to this emergency type?
    boolean isRemoteControlled();                       // true only for Drone
    void takeDamage(int amount);                        // humans: health -amount (min 0); robots: DAMAGED if
amount >= 30
    void recover(int amount);                           // humans: health +amount (max 100); robots: repaired
if amount >= 30
    void tick();                                        // one time unit: fatigue / battery update (D-09)
    String describe();                                  // one-line status for reports
}
```

```
public abstract class CrewMember implements Agent {
    public static final int UNUSABLE_HEALTH_AT_OR_BELOW = 20;
    public static final int FATIGUE_LIMIT = 100;
    public static final int FATIGUE_PER_TICK = 10;
    public static final int REST_PER_TICK = 5;

    private final String id;
    private final String name;
    private final Location home;
    private Location location;                  // starts at home
    private final Set<Skill> skills;            // copy into an EnumSet
    private int health = 100;                   // 0..100
```

```
    private int fatigue = 0;                  // 0..100
    private boolean assigned = false;

    protected CrewMember(String id, String name, Location home, Set<Skill> skills);
    public int getHealth();
    public int getFatigue();
    public Set<Skill> getSkills();            // unmodifiable
    public boolean hasSkill(Skill skill);
    public abstract String performDuty();     // polymorphism demo, see subclasses
    protected String lacksCapabilityReason();  // returns getName() + " does not possess the required
capability"
    // implements all Agent methods except getCapabilities(), getCannotPerformReason(), canHandle()
    // isRemoteControlled() -> false
}
```

Rules: `getUnusableReason()` → health ≤ 20: `name + " is too injured (health " + health + ")"`; fatigue = 100: `name + " is exhausted"`; else `null`. `tick()` per D-09. `takeDamage/recover` clamp 0..100; a negative amount → `IllegalArgumentException`. `assignToMission()` twice → `IllegalStateException`. `describe()` example: `"Dr. Layla [Scientist] @Habitat A health=100 fatigue=0 usable available"`.

```
public class Scientist extends CrewMember {
    public Scientist(String id, String name, Location home, Set<Skill> skills);
    // capabilities = {RESEARCH}
    // getCannotPerformReason: mission must require RESEARCH, else lacksCapabilityReason();
    //   if mission.getRequiredSkill()!=null and this lacks it -> name + " lacks the required skill " +
skill; else null
    // canHandle(any) -> false
    public String performDuty();              // "Conducting research."
}
public class Engineer extends CrewMember {
    public Engineer(String id, String name, Location home, Set<Skill> skills);
    // capabilities = {ENGINEERING}; performs mission if it requires ENGINEERING (else
lacksCapabilityReason())
    // canHandle: OXYGEN_LEAK, FIRE, ROVER_BREAKDOWN -> true; MEDICAL_EMERGENCY -> false
    public String performDuty();              // "Repairing equipment."
    public String repair(Mission mission);    // returns name + " is repairing for mission " +
mission.getId()   (the spec's association example)
}
public class MedicalOfficer extends CrewMember {
    public MedicalOfficer(String id, String name, Location home, Set<Skill> skills);
    // capabilities = {MEDICAL}; performs mission if it requires MEDICAL
    // canHandle: MEDICAL_EMERGENCY, FIRE -> true; others -> false
    public String performDuty();              // "Treating patients."
}
```

```
public abstract class Robot implements Agent {
    public static final int DAMAGE_THRESHOLD = 30;
    public static final int IDLE_RECHARGE = 1;

    private final String id;
    private final String name;
    private final Location home;
    private Location location;
    private int battery;                       // 0..100
    private RobotCondition condition = RobotCondition.OPERATIONAL;
    private boolean assigned = false;

    protected Robot(String id, String name, Location home, int battery);   // battery outside 0..100 ->
IllegalArgumentException
    public int getBattery();
    public void drain(int amount);             // min 0
    public void recharge(int amount);          // max 100
    public RobotCondition getCondition();
    public void damage();                      // condition = DAMAGED
    public void repair();                      // condition = OPERATIONAL
    protected abstract int getDrainPerTick();
    public abstract String performFunction();   // polymorphism demo
    protected String lacksCapabilityReason();   // name + " does not possess the required capability"
    protected String batteryReason(Mission m);  // null if battery >= m.getMinBatteryPercent(); else name +
" battery (" + battery + "%) is below the required " + min + "%"
    // implements all Agent methods except getCapabilities(), getCannotPerformReason(), canHandle()
    // getUnusableReason: DAMAGED -> name + " is damaged"; battery == 0 -> name + " battery is depleted";
```

```
else null
    // tick(): assigned -> drain(getDrainPerTick()); idle -> recharge(IDLE_RECHARGE)
    // isRemoteControlled() -> false (Drone overrides)
}
public class ExplorationRover extends Robot {
    private final List<Equipment> cargo = new ArrayList<>();     // AGGREGATION: equipment exists
independently
    public ExplorationRover(String id, String name, Location home, int battery);
    public ExplorationRover(String id, String name, Location home);       // battery 100
    public void loadEquipment(Equipment e);
    public List<Equipment> getCargo();                              // unmodifiable
    public boolean hasEquipment(String equipmentName);              // present AND functional
    // capabilities = {SURFACE_TRAVEL, VEHICLE}; drain 5
    // getCannotPerformReason: mission requires SURFACE_TRAVEL or VEHICLE else lacksCapabilityReason();
    //   then batteryReason(m); then each name in m.getRequiredEquipment(): missing -> name + " is missing
required equipment: " + eq
    // canHandle(any) -> false
    public String performFunction();            // "Exploring the surface."
}
public class RepairRobot extends Robot {
    public RepairRobot(String id, String name, Location home, int battery);
    public RepairRobot(String id, String name, Location home);
    // capabilities = {ROBOTIC_REPAIR}; drain 3; performs mission if it requires ROBOTIC_REPAIR, then
batteryReason
    // canHandle: OXYGEN_LEAK, ROVER_BREAKDOWN -> true; others false
    public String performFunction();            // "Repairing equipment inside facilities."
}
public class Drone extends Robot {
    public Drone(String id, String name, Location home, int battery);
    public Drone(String id, String name, Location home);
    // capabilities = {AERIAL_RECON, VEHICLE}; drain 8; performs mission if it requires VEHICLE or
AERIAL_RECON, then batteryReason
    // canHandle(any) -> false; isRemoteControlled() -> true
    public String performFunction();            // "Performing aerial reconnaissance."
}
```

`Robot.takeDamage(amount)`: `amount >= 30` → `damage()`. `Robot.recover(amount)`: `amount >= 30` → `repair()`. `Robot.describe()` example: `"Rover-1 [ExplorationRover] @Storage battery=100 OPERATIONAL usable available"`.

8.3 `marsops.facilities`

```
public final class ResourceInventory {                  // plain stock book-keeping
    public int get(ResourceType type);
    public void add(ResourceType type, int amount);      // amount <= 0 -> IllegalArgumentException
    public boolean remove(ResourceType type, int amount);// false and NO change if stock < amount
    public int removeUpTo(ResourceType type, int amount);// removes min(amount, stock), returns removed
    public int total();
    public Map<ResourceType, Integer> snapshot();        // copy, never null
}

public abstract class Facility implements Location {
    private final String name;
    private final int capacity;
    private FacilityStatus status = FacilityStatus.OPERATIONAL;
    private final ResourceInventory inventory = new ResourceInventory();   // COMPOSITION (created inside)
    private final List<Equipment> equipment = new ArrayList<>();           // AGGREGATION

    protected Facility(String name, int capacity);       // blank name or capacity <= 0 ->
IllegalArgumentException
    @Override public String getName();
    public int getCapacity();
    public FacilityStatus getStatus();
    public void setStatus(FacilityStatus status);
    public boolean isOperational();                      // status == OPERATIONAL
    public int getStored(ResourceType type);             // raw stock, ignores status
    public int getAvailable(ResourceType type);          // 0 unless OPERATIONAL, else getStored(type)
    public int getTotalStored();
    public boolean store(ResourceType type, int amount); // false (no change) if getTotalStored()+amount >
capacity
    public boolean withdraw(ResourceType type, int amount); // false (no change) if stock < amount
    public int lose(ResourceType type, int amount);      // loses min(amount, stock); returns actually
lost; NEVER negative
```

```
    public void addEquipment(Equipment e);
    public List<Equipment> getEquipment();               // unmodifiable
    public Equipment findEquipment(String equipmentName);// null if none
    public abstract String getKind();                    // "Habitat", "Laboratory", ...
    public String describe();                            // e.g. "Habitat A [Habitat] OPERATIONAL
OXYGEN=300 WATER=150"
    @Override public String toString();                  // the name
}

public class Habitat extends Facility {
    private final LifeSupportSystem lifeSupport;         // COMPOSITION: created in the constructor
    private boolean leaking = false;
    public Habitat(String name, int capacity);
    public LifeSupportSystem getLifeSupport();           // always the same object
    public boolean isLeaking();
    public void setLeaking(boolean leaking);
    public int loseOxygen(int amount);                   // = lose(OXYGEN, amount)
    public String getKind();                             // "Habitat"
}
public class LifeSupportSystem {
    private boolean functional = true;
    public boolean isFunctional();
    public void setFunctional(boolean functional);
    public String describe();                            // "Life support: functional" / "Life support:
FAILED"
}
public class Laboratory extends Facility {
    private boolean onFire = false;
    public Laboratory(String name, int capacity);
    public boolean isOnFire();
    public void setOnFire(boolean onFire);
    public void damageAllEquipment();                    // every Equipment.damage()
    public boolean hasFunctionalEquipment(String equipmentName);
    public String getKind();                             // "Laboratory"
}
public class StorageFacility extends Facility {
    public StorageFacility(String name, int capacity);
    public String getKind();                             // "Storage"
}
public class PowerStation extends Facility {
    private final int outputUnits;
    public PowerStation(String name, int capacity, int outputUnits);
    public int getOutputUnits();
    public String getKind();                             // "PowerStation"
}
public class CommunicationStation extends Facility {
    public CommunicationStation(String name, int capacity);
    public boolean isLinkUp();                           // = isOperational()
    public String getKind();                             // "CommunicationStation"
}
```

8.4 `marsops.environment` and `marsops.report`

```
public class Environment {
    private EnvironmentCondition condition = EnvironmentCondition.NORMAL;
    public EnvironmentCondition getCondition();
    public void setCondition(EnvironmentCondition condition);    // null -> IllegalArgumentException
    public boolean isNormal();
    public boolean isDustStorm();
    public boolean isCommunicationDisrupted();
    public String describe();                                    // "Environment: DUST_STORM"
}

public final class ColonyReport {
    private ColonyReport() { }
    /** Multi-line text report. Must contain what "Report format" below lists. */
    public static String build(MarsColony colony, int currentTime, List<Emergency> emergencies);
}
```

Report format (must contain, in this order): header `=== MARS-OPS COLONY STATUS REPORT (t=<currentTime>) ===` · environment line · **colony-wide resource totals** (one line per `ResourceType`, e.g. `OXYGEN: 550`) · facilities (one `describe()` line each) · agents (one `describe()` line each) · active mission

(`none` or its `describe()`) · mission counts (`completed=x failed=y`) · emergencies (one line each: id, type, state, reason; or `none`).

8.5 Your test file: `AgentsFacilitiesTests`

Rules for the file: `package marsops.tests;` · has `public static void main(String[] args)` · builds its own small fixture (copy what you need from Section 11) · one `Check.section(...)` per block · put the ID in each test name, e.g. `Check.isTrue("A-05 engineer handles FIRE", ...)` · ends with `Check.summary();` . Build missions with B's `ExplorationMission` (merge B's early push first).

ID	Check
A-01	New CrewMember: health 100, fatigue 0, not assigned, location = home, `isAvailable()` true
A-02	Capabilities: Scientist={RESEARCH}, Engineer={ENGINEERING}, MedicalOfficer={MEDICAL}, Rover={SURFACE_TRAVEL,VEHICLE}, RepairRobot={ROBOTIC_REPAIR}, Drone={AERIAL_RECON,VEHICLE}
A-03	Scientist with GEOLOGY: `getCannotPerformReason(explorationRequiringGeology)` is null; scientist without GEOLOGY gets a reason containing "skill"
A-04	MedicalOfficer vs Exploration mission → reason contains "required capability"; `canPerform` false
A-05	`canHandle` matrix: Engineer {OXYGEN_LEAK,FIRE,ROVER_BREAKDOWN}; Medical {MEDICAL_EMERGENCY,FIRE}; RepairRobot {OXYGEN_LEAK,ROVER_BREAKDOWN}; Scientist/Rover/Drone none (check all 6×4)
A-06	CrewMember `tick()` : assigned → fatigue +10, capped at 100; idle → fatigue −5, floor 0
A-07	`getUnusableReason()` : health 20 → non-null; health 21 → null; fatigue 100 → non-null; assigned agent → `isAvailable()` false but `isUsable()` true
A-08	`assignToMission()` twice → `IllegalStateException` ; after `release()` it works again
A-09	`takeDamage(200)` → health 0; `recover(500)` → health 100; negative amount → `IllegalArgumentException`
A-10	Robot ticks: assigned Rover −5, Drone −8, RepairRobot −3 battery; idle +1; never above 100 or below 0; battery 0 → unusable reason contains "battery"
A-11	Rover with battery 30 vs mission needing 40 → reason contains "battery"; rover without Spectrometer vs mission needing it → reason contains "equipment"; after `loadEquipment` it passes; damaged equipment does not count
A-12	`isRemoteControlled()` : true only for Drone
A-13	`damage()` → DAMAGED and unusable reason contains "damaged"; `repair()` restores; `takeDamage(30)` damages a robot, `takeDamage(29)` does not
A-14	Facility stock: `withdraw` more than stock → false and unchanged; `lose` more than stock → stock 0 and returns the actual amount; `store` beyond capacity → false and unchanged
A-15	`getAvailable` is 0 when status is DAMAGED / UNAVAILABLE / DESTROYED while `getStored` is unchanged; OPERATIONAL → equal
A-16	Habitat: `loseOxygen(1000)` on 300 → 0 (never negative); `getLifeSupport()` same object each call; two habitats have different LifeSupportSystem objects (composition)
A-17	Laboratory: `damageAllEquipment()` → `hasFunctionalEquipment("Microscope")` false
A-18	CommunicationStation `isLinkUp()` mirrors OPERATIONAL status
A-	Environment starts NORMAL; setter works; helper booleans match; `setCondition(null)` →

19	`IllegalArgumentException`
A-20	Polymorphism: a `List<Agent>` of all six types → every `describe()` non-empty and different; three `performDuty()` strings differ; three `performFunction()` strings differ; `Engineer.repair(mission)` contains the mission id
A-21	`ColonyReport.build(...)` (after B's colony is merged): contains `"OXYGEN: 550"` for the sample colony, the environment name, every agent name and every facility name

8.6 Done when (A)

- ☐ All A files compile; no public member differs from Section 8.
- ☐ `AgentsFacilitiesTests`: all 21 IDs pass.
- ☐ Javadoc on every class and public method (Section 14).
- ☐ Class diagram + Use Case diagram exported (Section 12), consistent with the code.
- ☐ README sections R1, R2, R3 written.
- ☐ Pushed to `main`; you can explain every class in Sections 8, 9 and 10.

---

9. MEMBER B — "Brain"

> ▶ **YOUR MISSION (B)** Build the *brain* of the colony: **missions**, **resource management** and the **colony rules** that decide whether a mission is accepted and how time moves it forward. **You also draw:** State Machine, Object diagram, Sequence A (mission), Sequence C (dust storm) · **You also write:** README sections R4, R5, R6. **Folders:** `missions`, `colony`, and `tests/ColonyMissionsTests.java`.

9.1 Do this first

☐ **1. Early push:** `Mission`, `AbstractMission`, the four mission classes, `SubmissionResult`.
☐ **2.** `AllocationPolicy`, `StorageFirstPolicy`, `ResourceManager` (write against A's `Facility` signature; A pushes it early).
☐ **3.** `MarsColony`.
☐ **4.** `ColonyMissionsTests` — use A's real agents and facilities once pushed. Until then a 10-line `FakeAgent implements Agent` inside your test file is fine.
☐ **5.** The four UML diagrams.

9.2 `marsops.missions`

```
public interface Mission {
    String getId();
    MissionType getType();
    MissionState getState();
    int getDurationUnits();
    int getProgressUnits();
    String getFailureReason();                          // null unless FAILED
    List<Agent> getAssignedAgents();                    // unmodifiable; empty until ASSIGNED
    List<Facility> getTargetFacilities();               // may be empty (e.g. exploration); unmodifiable
    ResourceRequirement getResourceRequirement();

    // --- what agents and the colony ask ---
    Set<Capability> getRequiredCapabilities();          // the TEAM must cover all of them
    Skill getRequiredSkill();                           // null if none
    int getMinBatteryPercent();                         // 0 if none
    List<String> getRequiredEquipment();                // equipment NAMES, may be empty
    boolean requiresCommunication();
    boolean requiresSurface();
    /** null if the mission may start now; otherwise the reason (environment, target facility state...). */
    String getStartBlockReason(EnvironmentCondition env);
    /** null if a mission already ASSIGNED / IN_PROGRESS may continue; otherwise why it must fail. */
    String getInProgressFailureReason(EnvironmentCondition env);

    // --- lifecycle: called ONLY by MarsColony ---
    void assign(List<Agent> agents);                    // CREATED -> ASSIGNED
    void start();                                       // ASSIGNED -> IN_PROGRESS
    void advance();                                     // IN_PROGRESS: progress+1; if progress >= duration
-> COMPLETED (calls onComplete())
    void fail(String reason);                           // CREATED/ASSIGNED/IN_PROGRESS -> FAILED
    boolean isFinished();                               // COMPLETED or FAILED
    String describe();                                  // "M-1 [EXPLORATION] IN_PROGRESS 2/3"
}
```

```
public abstract class AbstractMission implements Mission {
    private final String id;
    private final MissionType type;
    private final int durationUnits;                     // >= 1 else IllegalArgumentException
    private MissionState state = MissionState.CREATED;
    private int progressUnits = 0;
    private String failureReason;
    private final List<Agent> assignedAgents = new ArrayList<>();
    private final List<Facility> targetFacilities;
    private final ResourceRequirement resourceRequirement;

    protected AbstractMission(String id, MissionType type, int durationUnits,
                              ResourceRequirement resourceRequirement, List<Facility> targetFacilities);
```

```
    protected void onComplete() { }                    // hook (TEMPLATE METHOD): called once when
COMPLETED
    // implements: getId, getType, getState, getDurationUnits, getProgressUnits, getFailureReason,
getAssignedAgents,
    //             getTargetFacilities, getResourceRequirement, assign, start, advance, fail, isFinished,
describe
    // subclasses implement the 8 "what agents and the colony ask" methods
}
```

Lifecycle rules (tests B-01..B-03): wrong state → `IllegalStateException` (`assign` only from CREATED; `start` only from ASSIGNED; `advance` only from IN_PROGRESS; `fail` not from COMPLETED/FAILED). `fail(null or blank)` → `IllegalArgumentException`. `advance()` reaching `progress == duration` sets COMPLETED, then calls `onComplete()`.

```
public class ExplorationMission extends AbstractMission {
    public ExplorationMission(String id, String regionName, int durationUnits, int minBatteryPercent,
                              Skill requiredSkill, List<String> requiredEquipment, ResourceRequirement
resources);
    public String getRegionName();
    // type EXPLORATION · required capabilities {RESEARCH, SURFACE_TRAVEL} · requiresSurface=true ·
requiresCommunication=false · no target facilities
    // getStartBlockReason: DUST_STORM -> "Surface operations are not allowed during a dust storm";
otherwise null
    // getInProgressFailureReason: DUST_STORM -> "Dust storm: active rover mission aborted"; otherwise null
}
public class RepairMission extends AbstractMission {
    public RepairMission(String id, Facility target, int durationUnits, ResourceRequirement resources);
    // type REPAIR · required {ENGINEERING, ROBOTIC_REPAIR} · surface=false · communication=false · target
= [target]
    // getStartBlockReason: target DESTROYED -> "Facility <name> is destroyed and cannot be repaired";
otherwise null
    // getInProgressFailureReason: always null
    // onComplete(): target.setStatus(OPERATIONAL)
}
public class ResearchMission extends AbstractMission {
    public ResearchMission(String id, Laboratory lab, Skill requiredSkill, int durationUnits,
                           List<String> requiredEquipment, ResourceRequirement resources);
    // type RESEARCH · required {RESEARCH} · surface=false · communication=false · target = [lab]
    // getStartBlockReason: lab not OPERATIONAL -> "Laboratory <name> is not operational";
    //                       a required equipment name that the lab lacks or is damaged -> "Laboratory
<name> lacks working equipment: <eq>"
    // getInProgressFailureReason: lab no longer OPERATIONAL -> "Laboratory <name> became unavailable";
otherwise null
}
public class RescueMission extends AbstractMission {
    public RescueMission(String id, String strandedEntity, Location strandedAt, int durationUnits,
                         int minBatteryPercent, ResourceRequirement resources);
    // type RESCUE · required {VEHICLE, MEDICAL} · surface=false · communication=true
    // target facilities: [strandedAt] if it is a Facility, else empty
    // getStartBlockReason: COMMUNICATION_DISRUPTION -> "Communication disruption: rescue cannot be
coordinated"
    // getInProgressFailureReason: COMMUNICATION_DISRUPTION -> "Communication lost during rescue"
}
```

9.3 `marsops.colony`

```
public interface AllocationPolicy {
    /** Order in which facilities supply 'type'. Input: operational facilities in the order they were
added. */
    List<Facility> order(List<Facility> operationalFacilities, ResourceType type);
}
public class StorageFirstPolicy implements AllocationPolicy {      // D-01: StorageFacility objects first,
then the rest; keep original order inside each group
    public List<Facility> order(List<Facility> operationalFacilities, ResourceType type);
}

public final class SubmissionResult {
    public static SubmissionResult accepted(Mission mission);
    public static SubmissionResult rejected(Mission mission, String reason);
    public boolean isAccepted();
    public String getReason();              // "" when accepted
```

```
    public Mission getMission();
    @Override public String toString();      // accepted: "Mission M-1 accepted"; rejected: "Mission cannot
be assigned. Reason: <reason>"
}

public class ResourceManager {
    private final List<Facility> facilities;               // the colony's LIVE list (not a copy)
    private AllocationPolicy policy;
    public ResourceManager(List<Facility> facilities, AllocationPolicy policy);
    public void setPolicy(AllocationPolicy policy);
    public int getTotal(ResourceType type);                // sum of getAvailable over facilities (D-02)
    public Map<ResourceType, Integer> getAllTotals();      // all 7 types, in enum order
    public boolean canSupply(ResourceRequirement requirement);
    /** null if everything is available; else e.g. "Insufficient OXYGEN: need 600, available 550" (first
short type in enum order). */
    public String explainShortage(ResourceRequirement requirement);
    /** All-or-nothing: if explainShortage != null -> return false and change NOTHING. Otherwise withdraw
using the policy and return true. */
    public boolean consume(ResourceRequirement requirement);
}
```

```
public class MarsColony {
    private final String name;
    private final List<Agent> agents = new ArrayList<>();            // AGGREGATION (agents exist without
the colony)
    private final List<Facility> facilities = new ArrayList<>();     // AGGREGATION
    private final ResourceManager resourceManager;                   // COMPOSITION (created in constructor
with the live facilities list)
    private final Environment environment = new Environment();       // COMPOSITION
    private Mission activeMission;                                   // null when none
    private final List<Mission> missionHistory = new ArrayList<>();
    private final List<String> eventLog = new ArrayList<>();

    public MarsColony(String name, AllocationPolicy policy);
    public MarsColony(String name);                                  // uses StorageFirstPolicy

    public String getName();
    public void addAgent(Agent agent);                               // null / duplicate ->
IllegalArgumentException
    public void addFacility(Facility facility);                      // null / duplicate name ->
IllegalArgumentException
    public List<Agent> getAgents();                                  // unmodifiable view
    public List<Facility> getFacilities();                           // unmodifiable view
    public Facility findFacility(String name);                       // null if none
    public Agent findAgent(String id);                               // null if none
    public List<Agent> getAgentsAt(Location location);               // agents whose getLocation() ==
location
    public boolean hasOperationalCommunication();                    // any CommunicationStation that is
operational
    public Environment getEnvironment();
    public ResourceManager getResourceManager();

    public SubmissionResult submitMission(Mission mission, List<Agent> agents);   // rules S0–S8 below
    public Mission getActiveMission();
    public List<Mission> getMissionHistory();                        // unmodifiable; every accepted and
rejected mission, in order
    public boolean failActiveMission(String reason);                 // false if no active mission; else
fail + finish (see below) and true
    public void changeEnvironment(EnvironmentCondition condition);   // set it, then apply D-06 to the
active mission IMMEDIATELY
    public void tick();                                              // advance ONE time unit (see below)
    public void log(String message);
    public List<String> getEventLog();                               // unmodifiable
}
```

`submitMission` — run the checks in exactly this order; the first one that fails rejects the mission (`mission.fail(reason)`, add to history, return `rejected`). A rejected mission **never becomes active and consumes nothing**.

Step	Check	Reason text (tests compare with case-

		insensitive *contains*)
S0	`mission.getState() != CREATED`	`"Mission was already processed"` (do NOT call `fail`; just return rejected)
S1	an active mission exists	`"Another mission is already active (" + active.getId() + ")"`
S2	agents list null or empty	`"No agents were assigned to the mission"`
S3	`mission.getStartBlockReason(env)` ≠ null	that reason (e.g. the dust storm text)
S4	`mission.requiresCommunication()` and `!hasOperationalCommunication()`	`"No operational communication station"`
S5	per agent, in list order: not in colony → `name + " is not part of this colony"`; remote-controlled while COMMUNICATION_DISRUPTION → `name + " cannot be controlled during a communication disruption"`; `getCannotPerformReason(mission)` ≠ null → **that reason as is**; `!isAvailable()` → `name + " is not available (" + (assigned ? "assigned to a mission" : unusableReason) + ")"`	as written
S6	union of agent capabilities does not cover `getRequiredCapabilities()`	`"No assigned agent provides capability " + missing`
S7	`resourceManager.explainShortage(mission.getResourceRequirement())` ≠ null	that text
S8	**Accept:** `resourceManager.consume(...)`; `mission.assign(agents)`; every agent `assignToMission()`; `activeMission = mission`; add to history; `log(...)`	return `accepted(mission)`

`tick()` — exactly this order 1. Every agent in the colony: `agent.tick()`. 2. If there is an active mission: - **a.** state ASSIGNED → `mission.start()`; if `mission.requiresSurface()`, move every assigned agent that `hasCapability(SURFACE_TRAVEL)` to `MartianSurface.INSTANCE`. - **b.** `reason = mission.getInProgressFailureReason(env.getCondition())`; if not null → fail it (step 3). - **c.** else if any assigned agent has `getUnusableReason() != null` → fail with that reason (step 3). - **d.** else `mission.advance()`; if it is now COMPLETED → finish (step 3). 3. **Finish / fail:** (for failure: `mission.fail(reason)`), then every assigned agent `release()`; every assigned agent that `isUsable()` goes to `getHomeLocation()` (a damaged rover stays where it is); `activeMission = null`; `log(...)`.

`changeEnvironment(c)`: `environment.setCondition(c)`, then if an active mission exists and `getInProgressFailureReason(c) != null` → fail it as in step 3. `failActiveMission(reason)`: same finish step; used by C's `RoverBreakdown`.

9.4 Your test file: `ColonyMissionsTests`

Same rules as Section 8.5 (own fixture from Section 11, IDs in test names, `Check.summary()` at the end). Reason assertions use `Check.contains`.

ID	Check

B-01	Happy path of any mission: CREATED → `assign` → ASSIGNED → `start` → IN_PROGRESS → `advance()` × duration → COMPLETED; `getProgressUnits() == getDurationUnits()`; `isFinished()` true
B-02	Invalid transitions throw `IllegalStateException`: `start()` on CREATED, `advance()` on ASSIGNED, `assign()` twice, `fail()` on COMPLETED
B-03	`fail("x")` from CREATED, ASSIGNED and IN_PROGRESS → FAILED with reason "x"; `fail("")` / `fail(null)` → `IllegalArgumentException`; duration 0 → `IllegalArgumentException`
B-04	Requirement getters per mission type: capability sets, `requiresSurface` (only Exploration), `requiresCommunication` (only Rescue), target facilities (Exploration none, Repair/Research one)
B-05	`getStartBlockReason`: Exploration+DUST_STORM contains "dust storm", +NORMAL null; Rescue+COMMUNICATION_DISRUPTION non-null; Research with UNAVAILABLE lab non-null; Research with missing equipment non-null; Repair of DESTROYED facility non-null
B-06	`getInProgressFailureReason`: Exploration+DUST_STORM non-null; Research+DUST_STORM null; Rescue+COMMUNICATION_DISRUPTION non-null
B-07	Repair mission completion sets the target facility OPERATIONAL (`onComplete` hook)
B-08	`ResourceManager.getTotal` counts only OPERATIONAL facilities (set one DESTROYED and one UNAVAILABLE → their stock disappears from the total)
B-09	With Habitat 300 + Storage 200 oxygen (set the rest to 0): requirement 450 → `canSupply` true; 600 → false and `explainShortage` contains "OXYGEN", "need 600" and "available 500"
B-10	All-or-nothing: a requirement that is too big → `consume` false and every facility's `snapshot()` is unchanged
B-11	Multi-source + policy: sample colony, consume 450 oxygen → Storage 0, Habitat A 50, Lab 1 50 (Storage first, then Habitat A, then Lab)
B-12	Two-resource requirement where the second resource is short → first resource NOT consumed; totals never negative
B-13	Second mission while one is active → rejected, reason contains "already active", rejected mission FAILED and not active, history size 2
B-14	Unsuitable agent (Medical officer on Exploration) → rejected, reason contains "required capability", resources unchanged, active mission null
B-15	Oxygen requirement 600 (> 550) → rejected, reason contains "Insufficient OXYGEN", all totals unchanged and ≥ 0
B-16	Accepted mission: result accepted, state ASSIGNED, `getActiveMission()` is it, every agent `isAssigned()`, resources consumed exactly once
B-17	Ticks: after 1 tick IN_PROGRESS with progress 1; rover on `MartianSurface.INSTANCE` during; after `duration` ticks COMPLETED, agents released, active null, rover back home
B-18	Failed missions are not retried: submit the same FAILED mission again → rejected, reason contains "already processed"
B-19	Dust storm: `changeEnvironment(DUST_STORM)` then submit Exploration → rejected with "dust storm"; running Exploration + `changeEnvironment(DUST_STORM)` → FAILED immediately, agents released; a running Research mission is NOT affected
B-20	Communication disruption: submit a Drone mission → rejected "communication disruption"; Rescue mission → rejected
B-21	Rover battery reaching 0 mid-mission (start rover at 12%, drain 5/tick) → mission FAILED, reason contains "battery", agents released
B-22	`failActiveMission("x")` → true, mission FAILED "x"; with no active mission → false; a DAMAGED rover stays on the surface while usable agents go home
B-23	Team coverage: Scientist alone on Exploration → rejected, reason contains "SURFACE_TRAVEL"
B-	Unavailable agent (already assigned) → rejected, reason contains "not available"

```
24
```

9.5 Done when (B)

- ☐ All B files compile; signatures match Section 9.
- ☐ `ColonyMissionsTests`: all 24 IDs pass.
- ☐ State Machine, Object diagram, Sequence A and Sequence C drawn and consistent.
- ☐ README sections R4, R5, R6 written.
- ☐ Javadoc complete; pushed; you can explain every class in Sections 8, 9 and 10.

---

10. MEMBER C — "Crisis & Control"

> ▶ **YOUR MISSION (C)** Build everything that **goes wrong and drives the story**: the four **emergencies**, the **emergency manager**, the **simulation clock** and `Main` with the 5 demo scenarios. You also assemble the final README. **You also draw:** Sequence B (oxygen leak) · **You also write:** README sections R0, R7, R8, R9, R10. **Folders:** `emergencies`, `simulation`, `Main`, and `tests/EmergencySimulationTests.java`.

10.1 Do this first

☐ **1.** `Emergency`, `EmergencyManager`, `Simulation` (compile against the stubs).
☐ **2.** `OxygenLeak` first (it is the bonus mark), then `MedicalEmergency`, `RoverBreakdown`, `Fire`.
☐ **3.** Write `Main.buildSampleSimulation()` early using Section 11; write the 5 scenarios last (they need A's and B's code).
☐ **4.** `EmergencySimulationTests`, then Sequence B, then the README.

10.2 `marsops.emergencies`

```
public abstract class Emergency {
    private final String id;
    private final EmergencyType type;
    private final Location location;                        // a Facility or MartianSurface
    private EmergencyState state = EmergencyState.DETECTED;
    private String reason = "";
    private final List<Agent> responders = new ArrayList<>();

    protected Emergency(String id, EmergencyType type, Location location);
    public String getId();
    public EmergencyType getType();
    public Location getLocation();
    public EmergencyState getState();
    public String getReason();                              // why UNRESOLVED, or a short success note
    public List<Agent> getResponders();                     // unmodifiable
    public String describe();                               // "E-1 [OXYGEN_LEAK] RESOLVED - <reason>"

    public abstract List<Capability> getRequiredResponderCapabilities();   // D-10 (one distinct agent per
entry)
    public abstract ResourceRequirement getResponseResources();            // D-11
    protected abstract void applyInitialEffects(MarsColony colony);        // on detection
    protected abstract void applyResolution(MarsColony colony);            // after a successful response
    protected abstract void applyUnresolvedPenalty(MarsColony colony);     // when it ends UNRESOLVED
    protected String checkPreconditions(MarsColony colony) { return null; }// non-null = cannot be handled;
text becomes the reason
    protected boolean isExcluded(Agent agent) { return false; }            // agents that may never respond
(the patient)

    // package-private: used only by EmergencyManager
    void setState(EmergencyState state, String reason);
    void addResponder(Agent agent);
}

public class OxygenLeak extends Emergency {
    public OxygenLeak(String id, Habitat habitat, int oxygenLoss);         // oxygenLoss > 0
    // required: [ENGINEERING, ROBOTIC_REPAIR] · resources: 2 SPARE_PARTS + 5 BATTERY_POWER
    // initial: habitat.setLeaking(true); habitat.loseOxygen(oxygenLoss)
    // resolution: habitat.setLeaking(false)
    // unresolved penalty: habitat.loseOxygen(oxygenLoss) AGAIN (leaking stays true)
}
public class Fire extends Emergency {
    public Fire(String id, Laboratory lab);
    // required: [ENGINEERING, MEDICAL] · resources: 10 WATER + 1 MEDICAL_SUPPLIES + 3 BATTERY_POWER
    // initial (D-14): lab.setOnFire(true); setStatus(UNAVAILABLE); damageAllEquipment(); lab.lose(OXYGEN,
5);
    //                 every agent in colony.getAgentsAt(lab): takeDamage(30)
    // resolution: lab.setOnFire(false); lab.setStatus(DAMAGED); agents in lab recover(20)
    // unresolved penalty: lab.setStatus(DESTROYED)
```

```
}
public class RoverBreakdown extends Emergency {
    public RoverBreakdown(String id, ExplorationRover rover);              // location =
rover.getLocation() at construction
    // required: [ENGINEERING, ROBOTIC_REPAIR] · resources: 2 SPARE_PARTS + 5 BATTERY_POWER
    // initial (D-15): rover.damage(); if the active mission contains this rover ->
colony.failActiveMission("Rover <name> broke down")
    // resolution: rover.repair(); rover.setLocation(rover.getHomeLocation())
    // unresolved penalty: nothing (rover stays damaged where it is)
}
public class MedicalEmergency extends Emergency {
    public MedicalEmergency(String id, CrewMember patient, int severity);  // location =
patient.getLocation()
    // required: [MEDICAL] · resources: 2 MEDICAL_SUPPLIES + 5 OXYGEN
    // initial: patient.takeDamage(severity)
    // checkPreconditions: location is a Facility that is not operational -> "Facility <name> is not
operational"
    // isExcluded(agent): agent == patient
    // resolution: patient.recover(40)
    // unresolved penalty: nothing
}

public class EmergencyManager {
    private final MarsColony colony;
    private final List<Emergency> history = new ArrayList<>();
    public EmergencyManager(MarsColony colony);
    public Emergency handle(Emergency emergency);          // the flow below; returns the same object
    public List<Emergency> getHistory();                   // unmodifiable, in order
    public List<Emergency> getUnresolved();                // state UNRESOLVED only
}
```

`handle(e)` — exactly this order 1. Add to history, `log` it, `e.applyInitialEffects(colony)`. 2. `msg = e.checkPreconditions(colony)`; if not null → **UNRESOLVED(msg)**, go to step 7. 3. **Pick responders:** for each capability `c` in `e.getRequiredResponderCapabilities()` (in order) choose the first agent in `colony.getAgents()` that is not yet chosen, `!e.isExcluded(a)`, `a.isAvailable()`, `a.canHandle(e.getType())` and `a.hasCapability(c)`. If none → **UNRESOLVED("No available responder with capability " + c)**, go to step 7. *(Never test the concrete class of an agent here.)* 4. **Check resources:** `colony.getResourceManager().explainShortage(e.getResponseResources())`; if not null → **UNRESOLVED(text)**, go to step 7. Nothing is consumed. 5. **Respond:** for each chosen agent: `assignToMission()`, `setLocation(e.getLocation())`, `e.addResponder(a)`. Then `consume(e.getResponseResources())`, `e.applyResolution(colony)`, `e.setState(RESOLVED, "Repaired by " + names)`. 6. Release every responder (`release()`); they stay at the location (D-12). Go to step 8. 7. `e.applyUnresolvedPenalty(colony)`; `e.setState(UNRESOLVED, msg)`. (Never retried automatically.) 8. `log` the outcome and return `e`.

10.3 `marsops.simulation`

```
public class Simulation {
    private final MarsColony colony;
    private final EmergencyManager emergencyManager;           // COMPOSITION (created in constructor)
    private int currentTime = 0;
    private final List<ScheduledEnvChange> envChanges = new ArrayList<>();     // private static nested
class
    private final List<ScheduledEmergency> emergencies = new ArrayList<>();    // private static nested
class

    public Simulation(MarsColony colony);
    public MarsColony getColony();
    public EmergencyManager getEmergencyManager();
    public int getCurrentTime();
    public void scheduleEnvironmentChange(int atTime, EnvironmentCondition condition);   // atTime >= 1
    public void scheduleEmergency(int atTime, Emergency emergency);                     // atTime >= 1
    public void triggerEmergency(Emergency emergency);     // manual trigger NOW (the spec says "manually
triggered"): emergencyManager.handle(...)
    public void advanceTime(int units);                    // units >= 0 else IllegalArgumentException;
calls advanceOneUnit() units times
    public void advanceOneUnit();                          // order below
    public String report();                                // ColonyReport.build(colony, currentTime,
emergencyManager.getHistory())
```

```
}
```

`advanceOneUnit()` order: (1) `currentTime++` (2) every scheduled env change with `atTime == currentTime` → `colony.changeEnvironment(...)` (3) `colony.tick()` (4) every scheduled emergency with `atTime == currentTime` → `emergencyManager.handle(...)`. Each scheduled item fires once.

10.4 `marsops.Main`

```
public class Main {
    public static void main(String[] args);                // runs the 5 scenarios in order, prints a final
"ALL SCENARIOS PASSED" or lists failures
    public static Simulation buildSampleSimulation();      // builds the Section 11 sample colony inside a
new Simulation
    public static boolean scenario1_successfulExploration();
    public static boolean scenario2_missionRejection();
    public static boolean scenario3_resourceShortage();
    public static boolean scenario4_emergencyHandling();   // 4a repaired, 4b unresolved
    public static boolean scenario5_environmentalEffect(); // 5a start blocked, 5b running mission aborted
}
```

Each scenario method builds a fresh sample simulation, **prints** a heading, the report **before** and **after** and the outcome text, then **returns `true` only if the observed result matches the expectations below** (so tests can call them).

Scenario	Steps	Must be true
1 Successful Exploration	Exploration "M-1", region "Valles", duration 3, min battery 40, skill GEOLOGY, equipment ["Spectrometer"], resources 10 FUEL + 5 BATTERY_POWER; agents [Dr. Layla, Rover-1]; submit; `advanceTime(3)`	Accepted; ASSIGNED after submit; COMPLETED after 3 ticks; FUEL total drops by 10 (140 → 130); BATTERY_POWER total drops by 5; both agents released and home; Rover-1 battery 85; Dr. Layla fatigue 30
2 Mission Rejection	Same exploration but agents [Dr. Salma (medical), Rover-1]	Rejected; prints `Mission cannot be assigned.` and `Reason: Dr. Salma does not possess the required capability`; mission FAILED; `getActiveMission()` null; resources unchanged
3 Resource Shortage	Research mission "M-3" in "Lab 1", skill CHEMISTRY, duration 2, equipment ["Microscope"], resources 600 OXYGEN; agents [Dr. Layla]	Rejected, reason contains "Insufficient OXYGEN"; every total unchanged and ≥ 0
4 Emergency Handling	**4a** OxygenLeak "E-1" in Habitat A, loss 80, via `triggerEmergency`. **4b** fresh colony: put Eng. Omar into an active Repair/Research mission first (or damage Fixer-1), then trigger the same leak	**4a:** RESOLVED; Habitat A oxygen 300 → 220; colony oxygen 550 → 470; SPARE_PARTS −2, BATTERY_POWER −5; Eng. Omar and Fixer-1 located at Habitat A and released; habitat not leaking. **4b:** UNRESOLVED with non-empty reason; Habitat A oxygen 300 → 140 (loss 160); no spare parts or battery consumed
5 Environmental Effect	**5a** `changeEnvironment(DUST_STORM)`, then submit the scenario-1 mission. **5b** fresh colony, accept the scenario-1 mission, `advanceTime(1)`, then `changeEnvironment(DUST_STORM)`	**5a:** rejected; prints `Exploration Mission cannot start.` and `Reason: Surface operations are not allowed during a dust storm`. **5b:** mission FAILED, reason contains "Dust storm", agents released

10.5 Your test file: `EmergencySimulationTests`

Same rules as Section 8.5. Build the sample colony with your own helper (or `Main.buildSampleSimulation()` once it exists).

ID	Check
C-01	For each of the 4 emergency classes: `getType()`, `getRequiredResponderCapabilities()` and `getResponseResources()` match D-10 / D-11
C-02	OxygenLeak detection: Habitat 300, loss 80 → 220; loss 1000 on 300 → 0 (never negative)
C-03	OxygenLeak repaired: RESOLVED; total oxygen loss exactly 80 (not 160); `isLeaking()` false; SPARE_PARTS −2 and BATTERY_POWER −5; both responders `isAssigned()` false and located at the habitat
C-04	No engineer available (assigned to a mission): UNRESOLVED, reason contains "ENGINEERING"; oxygen loss exactly 160; spare parts and battery unchanged; `getUnresolved().size() == 1`; after `simulation.advanceTime(2)` nothing was retried (history size and oxygen unchanged)
C-05	SPARE_PARTS set to 1 (needs 2): UNRESOLVED, reason contains "SPARE_PARTS"; nothing consumed; loss 160
C-06	Responder selection uses `canHandle`: a colony where the only free human is a Scientist → no human responder, UNRESOLVED; the Scientist is never dispatched
C-07	A responder busy in the active mission is skipped; a second free engineer is chosen instead
C-08	Emergency does not interrupt the active mission: a mission IN_PROGRESS stays IN_PROGRESS after an oxygen leak
C-09	Fire resolved: lab DAMAGED, `isOnFire()` false, equipment damaged, the agent that was in the lab lost 30 then recovered 20. Fire unresolved (no medic): lab DESTROYED and its stock no longer in colony totals
C-10	RoverBreakdown: rover DAMAGED and still on `MartianSurface.INSTANCE`; the active exploration mission FAILED with reason containing "broke down"; responders dispatched to the surface; after resolution rover OPERATIONAL and at home
C-11	MedicalEmergency: health lower after detection and higher after treatment; 2 MEDICAL_SUPPLIES and 5 OXYGEN consumed; no medic → UNRESOLVED; non-operational facility → UNRESOLVED with "not operational"; a medical-officer patient is never their own responder
C-12	`advanceTime(3)` → `getCurrentTime() == 3`; a 3-unit mission accepted before is COMPLETED; `advanceTime(-1)` → `IllegalArgumentException`
C-13	`scheduleEnvironmentChange(2, DUST_STORM)`: after 1 unit NORMAL, after 2 units DUST_STORM, stays DUST_STORM
C-14	`scheduleEmergency(2, leak)`: nothing after 1 unit; after 2 units the habitat lost oxygen; after 5 units it was handled exactly once (history size 1)
C-15	`triggerEmergency` handles immediately without advancing time (`getCurrentTime()` unchanged)
C-16	`EmergencyManager.getHistory()` keeps order; `getUnresolved()` contains only UNRESOLVED ones
C-17	`Main.scenario1_successfulExploration()` returns true
C-18	`Main.scenario2_missionRejection()` returns true
C-19	`Main.scenario3_resourceShortage()` returns true
C-20	`Main.scenario4_emergencyHandling()` and `Main.scenario5_environmentalEffect()` both return true

10.6 Done when (C)

☐ All C files compile; signatures match Section 10.
☐ `EmergencySimulationTests`: all 20 IDs pass; `java -cp out marsops.Main` prints all 5 scenarios.
☐ Sequence B drawn and consistent.
☐ README sections R0, R7–R10 written; the README is assembled and proofread.
☐ Javadoc complete; pushed; you can explain every class in Sections 8, 9 and 10.

---

11. Sample colony (numbers for tests and demos)

Everyone uses **these exact numbers** so tests and demos agree. Add the facilities to the colony **in this order** (the order matters for the allocation policy).

#	Facility	Class	Capacity	Stock	Equipment
1	`Habitat A`	Habitat	1000	OXYGEN 300, WATER 150, FOOD 150	—
2	`Storage`	StorageFacility	2000	OXYGEN 200, WATER 200, FOOD 200, FUEL 100, BATTERY_POWER 100, MEDICAL_SUPPLIES 20, SPARE_PARTS 10	—
3	`Lab 1`	Laboratory	500	OXYGEN 50, BATTERY_POWER 20	`Microscope`
4	`Power 1`	PowerStation (output 100)	500	FUEL 40, BATTERY_POWER 100	—
5	`Comm 1`	CommunicationStation	200	—	—

Totals: OXYGEN 550 · WATER 350 · FOOD 350 · FUEL 140 · BATTERY_POWER 220 · MEDICAL_SUPPLIES 20 · SPARE_PARTS 10

Agent	Id	Class	Home	Details
Dr. Layla	H-01	Scientist	Habitat A	skills GEOLOGY, CHEMISTRY
Eng. Omar	H-02	Engineer	Habitat A	skills ENGINEERING
Dr. Salma	H-03	MedicalOfficer	Habitat A	skills MEDICINE
Rover-1	R-01	ExplorationRover	Storage	battery 100, cargo: Equipment("Spectrometer")
Fixer-1	R-02	RepairRobot	Storage	battery 100
Drone-1	R-03	Drone	Storage	battery 100

Colony name `Mars Base Alpha` · policy `StorageFirstPolicy` · environment NORMAL · time 0.

---

12. UML guide — from zero

▶ **WHO DRAWS WHAT A:** Class diagram (12.2) + Use Case diagram (12.6) · **B:** State Machine (12.4) + Object diagram (12.3) + Sequence A and C (12.5) · **C:** Sequence B (12.5). Use **draw.io** (diagrams.net): *File → New → blank*, then the UML shape library (left panel → *More Shapes... → UML*). Save the editable `.drawio` file in `uml/` and export PNG or PDF (*File → Export as*). One diagram = one page. **Draw from this playbook or the final code** so diagrams match the Java.

12.1 Notation cheat sheet

What	Looks like	Java signal	MARS-OPS example
Class box	3 compartments: **Name** / **attributes** / **methods**	`class`	`Scientist`
Visibility	`+` public · `-` private · `#` protected	modifier	`-health: int` · `+canHandle(t: EmergencyType): boolean`
Abstract class / interface	name in *italics* / `«interface»` above the name	`abstract` / `interface`	`CrewMember`, `«interface» Agent`
Enumeration	`«enumeration»` + literals	`enum`	`MissionState`
Association	plain line (optional arrow)	one class *uses* another (field or parameter)	`Engineer` — `Mission`
Aggregation	line with **hollow** diamond at the whole	whole has parts that live on their own; parts come from outside	`MarsColony ◇– Agent`
Composition	line with **filled** diamond at the whole	whole creates and owns the part	`Habitat ◆– LifeSupportSystem`
Inheritance	solid line, **hollow triangle** → parent	`extends`	`Engineer –▷ CrewMember`
Realization	**dashed** line, hollow triangle → interface	`implements`	`CrewMember ⇢▷ Agent`
Multiplicity	numbers at the line ends	`1`, `0..1`, `*`	`MarsColony 1 ◇– * Agent`

12.2 Class diagram (A) — must contain

- All classes, interfaces and enums from Sections 7–10 with attributes (type), methods (parameters, return type), visibility, multiplicities, and a one-line responsibility note per major class.
- **Inheritance:** `CrewMember` → Scientist / Engineer / MedicalOfficer · `Robot` → ExplorationRover / RepairRobot / Drone · `Facility` → Habitat / Laboratory / StorageFacility / PowerStation / CommunicationStation · `AbstractMission` → 4 missions · `Emergency` → 4 emergencies. **Realization:** `CrewMember`, `Robot` ⇢ `Agent` · `AbstractMission` ⇢ `Mission` · `StorageFirstPolicy` ⇢ `AllocationPolicy`.
- **Composition (filled):** `Habitat ◆ LifeSupportSystem` · `Facility ◆ ResourceInventory` · `MarsColony ◆ ResourceManager` · `MarsColony ◆ Environment` · `Simulation ◆ EmergencyManager`.
- **Aggregation (hollow):** `MarsColony ◇ Agent` (1 to *) · `MarsColony ◇ Facility` (1 to *) · `ExplorationRover ◇ Equipment` · `Facility ◇ Equipment`.
- **Association (plain):** `Engineer – Mission` (the `repair(Mission)` method) · `AbstractMission – Agent` · `AbstractMission – Facility` · `Emergency – Agent` (responders) · `Emergency – Location` · `Simulation – MarsColony` · `ResourceManager – AllocationPolicy` · `MarsColony – Mission` (active, 0..1).
- *Tip:* group it in four areas (Agents · Facilities · Missions/Colony · Emergencies/Simulation). You may split it over 2 pages, but keep one overview page.

12.3 Object diagram (B) — snapshot "immediately before exploration"

(Scenario 1)

An object box is `name : ClassName` (underlined) with `attribute = value` lines. Draw these objects and links: - `colony : MarsColony` (name = "Mars Base Alpha", activeMission = null) - `habitatA : Habitat` (oxygen = 300, status = OPERATIONAL), linked ◆ to `lss : LifeSupportSystem` (functional = true) - `storage : StorageFacility` (oxygen = 200, fuel = 100, batteryPower = 100) - `layla : Scientist` (health = 100, fatigue = 0, skills = {GEOLOGY, CHEMISTRY}, assigned = false) - `rover1 : ExplorationRover` (battery = 100, condition = OPERATIONAL, location = storage), linked ◇ to `spectro : Equipment` (name = "Spectrometer", functional = true) - `m1 : ExplorationMission` (state = CREATED, duration = 3, progress = 0, minBattery = 40) - `env : Environment` (condition = NORMAL) and `rm : ResourceManager` - **Links:** colony◇layla, colony◇rover1, colony◇habitatA, colony◇storage, colony◆rm, colony◆env, rover1◇spectro, habitatA◆lss. Add a note: "`m1` gets linked to `layla` and `rover1` only after acceptance". - Every link must exist in the class diagram.

12.4 State machine diagram (B) — mission lifecycle

States: `CREATED`, `ASSIGNED`, `IN_PROGRESS`, `COMPLETED`, `FAILED`. Initial ● → `CREATED`; final ◉ after COMPLETED and after FAILED. Transitions (event [guard] / action):

1. `CREATED → ASSIGNED`: `submitMission` [no active mission ∧ environment allows ∧ agents capable and available ∧ team covers capabilities ∧ resources sufficient] / consume resources, assign agents
2. `CREATED → FAILED`: `submitMission` [any check above fails] / store failure reason (never active)
3. `ASSIGNED → IN_PROGRESS`: `tick` / start, position surface agents
4. `IN_PROGRESS → IN_PROGRESS`: `tick` [progress < duration ∧ no failure] / progress++ (a self-loop)
5. `IN_PROGRESS → COMPLETED`: `tick` [progress = duration] / `onComplete()`, release agents
6. `ASSIGNED → FAILED` and `IN_PROGRESS → FAILED`: `tick`, `changeEnvironment` or `failActiveMission` [dust storm for rover missions ∨ communication lost ∨ agent unusable (battery, damage, fatigue) ∨ rover breakdown ∨ lab unavailable] / store reason, release agents

Add a note: "FAILED and COMPLETED are final — a failed mission is never retried; create a new Mission."

12.5 Sequence diagrams

Notation: **lifelines** are boxes with dashed lines going down · solid arrow = call · dashed arrow = return · **`alt`** fragment with `[condition]` / `[else]` = success vs. failure · **`loop`** fragment = repeated steps.

Sequence A — Mission submission and execution (B). Participants: `Operator`, `MarsColony`, `Mission`, `Agent`, `ResourceManager`, `Simulation`, `Environment`. - `Operator → MarsColony: submitMission(mission, agents)` - `MarsColony`: is another mission active? → `MarsColony → Mission: getStartBlockReason(env)` - `loop` each agent: `MarsColony → Agent: getCannotPerformReason(mission)`, `isAvailable()` - `MarsColony → ResourceManager: explainShortage(req)` - **`alt` [any check fails]:** `MarsColony → Mission: fail(reason)`; return `rejected(reason)`; note "mission never becomes active" - **`[else]`:** `MarsColony → ResourceManager: consume(req)`; `MarsColony → Mission: assign(agents)`; `loop`: `Agent: assignToMission()`; state ASSIGNED; return `accepted` - Then `Operator → Simulation: advanceTime(n)` → `loop n: Simulation → MarsColony: tick()` → `MarsColony → Agent: tick()`, `MarsColony → Mission: start() / advance()` - **`alt` [progress = duration]:** Mission COMPLETED; `MarsColony → Agent: release()`. **`[failure]`:** `Mission: fail(reason)`; release agents.

Sequence B — Oxygen leak (C). Participants: `EmergencyOperator`, `Simulation`, `EmergencyManager`, `OxygenLeak`, `Habitat`, `MarsColony`, `Agent` (engineer / robot), `ResourceManager`. - `EmergencyOperator → Simulation: triggerEmergency(leak)` → `EmergencyManager: handle(leak)` - `OxygenLeak: applyInitialEffects` → `Habitat: loseOxygen(loss)` - `loop` each required capability: `MarsColony: getAgents()`; `Agent: isAvailable(), canHandle(OXYGEN_LEAK)` - **`alt` [responder missing]:** `OxygenLeak: applyUnresolvedPenalty → Habitat: loseOxygen(loss)` (second deduction); state UNRESOLVED(reason). - **`alt` [resources short]:** `ResourceManager: explainShortage` → same unresolved branch; nothing consumed. - **`[else]`:** `Agent: assignToMission(), setLocation(habitat)`; `ResourceManager: consume`; `OxygenLeak: applyResolution → Habitat: setLeaking(false)`; state RESOLVED; `Agent: release()`.

Sequence C — Environmental restriction (B). Participants: `Operator`, `MarsColony`, `Environment`, `ExplorationMission`. - `Operator → MarsColony: changeEnvironment(DUST_STORM)` → `Environment: setCondition` - `Operator → MarsColony: submitMission(explorationMission, agents)` → `MarsColony → ExplorationMission: getStartBlockReason(DUST_STORM)`; returns "Surface operations are not allowed during a dust storm" - **`alt` [reason ≠ null]:** `ExplorationMission: fail(reason)`; return `rejected(reason)`; note "mission never active; no resources consumed".

12.6 Use case diagram (A)

Draw a system boundary box titled **MARS-OPS**. Stick-figure actors: **Colony Operator** (left) and **Emergency Operator** (right). Use cases (ovals):

- **Colony Operator:** Manage crew and robotic units · Manage facilities and equipment · Monitor resources · Generate colony status report · Submit mission · Execute and monitor missions · Advance simulation time · Monitor environmental conditions · Inspect mission and emergency outcomes.
- **Emergency Operator:** Trigger and handle emergency · Inspect mission and emergency outcomes · Monitor resources.
- **`«include»`** (always needed): *Submit mission* includes *Validate agents* and *Check resources* · *Generate colony status report* includes *Monitor resources* · *Execute and monitor missions* includes *Advance simulation time*.
- **`«extend»`** (only sometimes; the arrow points from the extension to the base): *Reject mission* extends *Submit mission* [validation fails] · *Trigger and handle emergency* extends *Advance simulation time* [an emergency is scheduled for this time unit].

12.7 Consistency check before you export

- Every class and method in the class diagram exists in Sections 8–10.
- States in the state diagram equal `MissionState`.
- Messages in sequence diagrams are real method names.
- Values in the object diagram are possible in the code.

---

13. README — who writes what

Each member writes their sections as plain Markdown; C assembles and proofreads.

#	Section	Author
R0	Title, team names, how to compile and run (Section 4.2), how to run each scenario (`java -cp out marsops.Main`) and each test file	C
R1	System description and main components (one paragraph per package)	A
R2	Relationships: for association, aggregation, composition and inheritance — **why chosen, how shown in UML, how shown in Java** (use 12.2 as the list)	A
R3	Agents and facilities decisions (D-08, D-09, D-18); polymorphism and capability design (`canPerform` / `canHandle`)	A
R4	Resource allocation rules (D-01, D-02, D-03)	B
R5	Mission states and rules (D-04, D-05, D-07) with the S0–S8 table	B
R6	Environmental effects, **including what happens to missions already in progress** (D-06)	B
R7	Emergency rules and requirements for all four emergencies (D-10 … D-17)	C
R8	Time and simulation (`advanceTime`, order of events)	C
R9	Demonstration scenarios — what each prints and how to run it	C

R10	Other decisions left open by the spec + testing instructions	C

14. Javadoc and clean-code habits

14.1 Javadoc — how

Javadoc is a special comment `/** ... */` placed **directly above** a class, field or method. Tools turn it into HTML.

- **Every class and interface:** one sentence saying what it is + `@author` (your letter).
- **Every public method and constructor:** first sentence = what it does; then `@param` for each parameter, `@return` if not void, `@throws` for exceptions you throw.
- Explain **why** / the rule, not what the code obviously does.
- Private methods: a normal `//` comment is enough.

```
/**
 * Removes resources from this facility without ever going below zero.
 *
 * @param type   the resource to remove
 * @param amount how many units to remove (must be positive)
 * @return the number of units actually removed (can be less than {@code amount})
 * @throws IllegalArgumentException if {@code amount} is not positive
 */
public int lose(ResourceType type, int amount) { ... }

/**
 * A habitat: a facility where the crew live. It owns its life-support system (composition).
 *
 * @author A
 */
public class Habitat extends Facility { ... }
```

Tip: in IntelliJ / VS Code type `/**` above a method and press Enter — the `@param` lines are created for you. **Generate the HTML docs** (optional, great for the discussion): `javadoc -d docs/api -sourcepath src -subpackages marsops`

14.2 Clean-code checklist

1. **Encapsulation:** all fields `private`; change state only through methods that check the rules (no negative resources, health 0..100).
2. **Never return internal lists** — return `Collections.unmodifiableList(...)`.
3. **Program to interfaces:** variables of type `List<Agent>`, `Location`, `Mission` — not `ArrayList` or concrete classes.
4. **Ask, don't check the type:** `agent.canPerform(mission)`, `agent.canHandle(type)` — never `instanceof` to decide capability.
5. **One class = one job:** the colony coordinates, `ResourceManager` counts resources, an `Emergency` knows its own effects, `Simulation` only moves time.
6. **Fail fast:** wrong arguments → `IllegalArgumentException`; wrong state → `IllegalStateException`, with a clear message.
7. **No magic numbers:** use named `public static final` constants (e.g. `FATIGUE_PER_TICK`).
8. **No printing in model classes:** return Strings; `Main` and `ColonyReport` print.
9. **Names:** classes `PascalCase`, methods and fields `camelCase`, constants `UPPER_SNAKE_CASE`; booleans read like questions (`isAvailable`, `hasEquipment`).
10. `@Override` on every overridden method; `final` on fields that never change.
11. **Small methods** (aim for ≤ 30 lines); extract private helpers.
12. **Formatting:** 4 spaces, one class per file, ~120 characters per line max, no unused imports, no dead code.
13. **Commit messages:** `"B: add ResourceManager.consume"` — who + what.

14. **Test as you go:** after each class, run your test file.

---

15. Finish line

15.1 Integration checklist (all three, once everyone is done)

☐ `git pull`; clean compile from scratch (delete `out/`, compile again).
☐ Run all three test files — **0 failures**.
☐ Run `Main` — all 5 scenarios pass and print clearly.
☐ UML consistent with code (12.7); diagrams exported readable; `.drawio` sources saved in `uml/`.
☐ README complete (R0–R10) and includes the assumptions D-01 … D-18.
☐ Javadoc present everywhere.

15.2 Submission (one ZIP, online, on time — late is not allowed)

1. Folder name `StudentID-Prog-II-Lab5` (use the format the course announces; with 3 members agree which IDs go in the name).
2. Contains: `src/`, `README.md`, `uml/`, plus anything the course asks for.
3. Zip the **folder**; unzip it somewhere else, compile and run to verify.
4. **Everyone attends the discussion in their own timeslot.**

15.3 Discussion prep — each of you must be able to answer

1. Why is `Mission` an interface and `AbstractMission` a class? (template method `onComplete`)
2. Where did you use inheritance, polymorphism, abstraction, encapsulation? Give one class for each.
3. Why is `Habitat ◆ LifeSupportSystem` composition but `MarsColony ◇ Agent` aggregation?
4. Why do missions ask `canPerform` instead of checking types?
5. Walk through `submitMission` S0–S8. Why consume resources only at acceptance?
6. What does "all-or-nothing" mean and how is it implemented?
7. Why does an unresolved oxygen leak cost twice as much oxygen?
8. What happens to a running exploration mission when a dust storm starts?
9. How does the state machine diagram map to `AbstractMission`?
10. Show the object diagram's values in the running program (use the report).

Appendix A — Instructions for AI assistants

You are helping ONE member of a 3-person team build part of a Java project. **This document is the only source of truth. Follow it exactly.**

Step 1 — Find out who you are helping. If the human has not said which member they are, ask: "*Which member are you: A, B or C?*" Then read Section 2 (ownership) and only that member's section (8, 9 or 10).

Step 2 — Rules you must obey

1. **Only create or edit files that your member owns** (Section 2 and the member's section). Never write code for another member's files. If you need something from another member, use the signature in this document. If their code is not merged yet, write a small *local fake* in the test file only (never commit a fake into another member's package).
2. **Signatures are frozen.** Class names, package names, field names, constructors, method names, parameter types and return types must match this document exactly. Private helper methods are free. Do **not** add, remove or rename public members.
3. **Never edit this document unless the human explicitly instructs you to.** Before changing it you must first ask in chat: "*Do you really want me to change the master doc?*" and wait for a clear "yes". If you think the document has a mistake, **say so in chat and propose the change** — do not apply it.
4. If the human asks for code that contradicts this document, point out the conflict first and let the human decide.
5. Write complete files (no `...` placeholders), with Javadoc as described in Section 14.
6. Always finish by telling the human which **test IDs** (Sections 8–10) the code should satisfy.
7. Do not invent features outside this document. Do not use `instanceof` / `getClass()` to decide whether an agent can do a mission or emergency — always ask the agent (`canPerform`, `canHandle`).
8. Keep explanations simple and short. Give small examples.

Appendix B — The project in one paragraph

A Mars colony (`MarsColony`) owns agents (humans and robots) and facilities (habitats, labs, storage…). Resources (oxygen, water, …) are stored inside facilities but managed colony-wide. The colony accepts **one active mission at a time**; a mission is accepted only if the agents are capable and available, the environment allows it and resources are sufficient — otherwise it becomes `FAILED` immediately with a clear reason. A `Simulation` advances time in units: missions progress, batteries drain, fatigue grows, and scheduled environment changes and emergencies happen. Emergencies (oxygen leak, fire, rover breakdown, medical) are handled by asking agents whether they can respond.

Appendix C — Spec coverage matrix (what proves each requirement)

Spec section	Requirement	Where	Proved by
3.1	Personnel with name, id, health, location, skills, fatigue, availability	`CrewMember` + 3 subclasses	A-01, A-06, A-07
3.2	Robots with battery, damage, location, availability	`Robot` + 3 subclasses	A-10, A-13
4	Facilities with location, capacity, status, resources, equipment	`Facility` family	A-14 … A-18
5, 5.1	Resources tracked; colony totals; multiple sources; all-or-nothing; policy documented	`ResourceManager`, D-01 … D-03	B-08 … B-12, Scenario 3

6, 6.1	Mission types; ask the agent (`canPerform`)	`Agent`, `Mission`	A-03, A-04, B-04, B-14
6.2	One active mission; rejected → FAILED with reason; failed not retried; agents released	`MarsColony.submitMission`	B-13 … B-18, B-24
6.3	Colony owns missions; facilities do not	`MarsColony`	B-16, class diagram
7	Lifecycle, duration, failure causes	`AbstractMission`, `tick`	B-01 … B-03, B-17, B-21
8, 8.1, 8.2	Dust storm and communication disruption; README on in-progress behaviour	missions + `changeEnvironment`, D-06	B-05, B-06, B-19, B-20, Scenario 5
9.1	Locations, responders, availability, `canHandle`, unresolved rules	`EmergencyManager.handle`	C-04 … C-07
9.2	Oxygen leak, double loss if unresolved, never below zero	`OxygenLeak`, `Habitat.loseOxygen`	A-16, C-02 … C-05, Scenario 4
9.3	Fire consequences	`Fire`, D-14	C-09
9.4	Rover breakdown	`RoverBreakdown`, D-15	C-10
9.5	Medical emergency	`MedicalEmergency`, D-16	C-11
10	Simulation and time	`Simulation`	C-12 … C-15
11.1	Association, aggregation, composition, inheritance	Section 12.2	A-16, A-20, class diagram
11.2 – 11.6	5 UML diagram types	Section 12	diagram files
12	5 demonstration scenarios	`Main`	C-17 … C-20
13	Deliverables: code, UML, README, Main	Sections 13, 15	integration checklist

End of playbook. Do not change this file unless the human explicitly instructs it and confirms in chat.